

Generazione di password sicure in Python – W6D4

Ambiente di sviluppo: Kali Linux, Visual Studio Code (terminale integrato)

File del progetto: W6D4.py, facoltativo.py

Obiettivo dell'esercizio

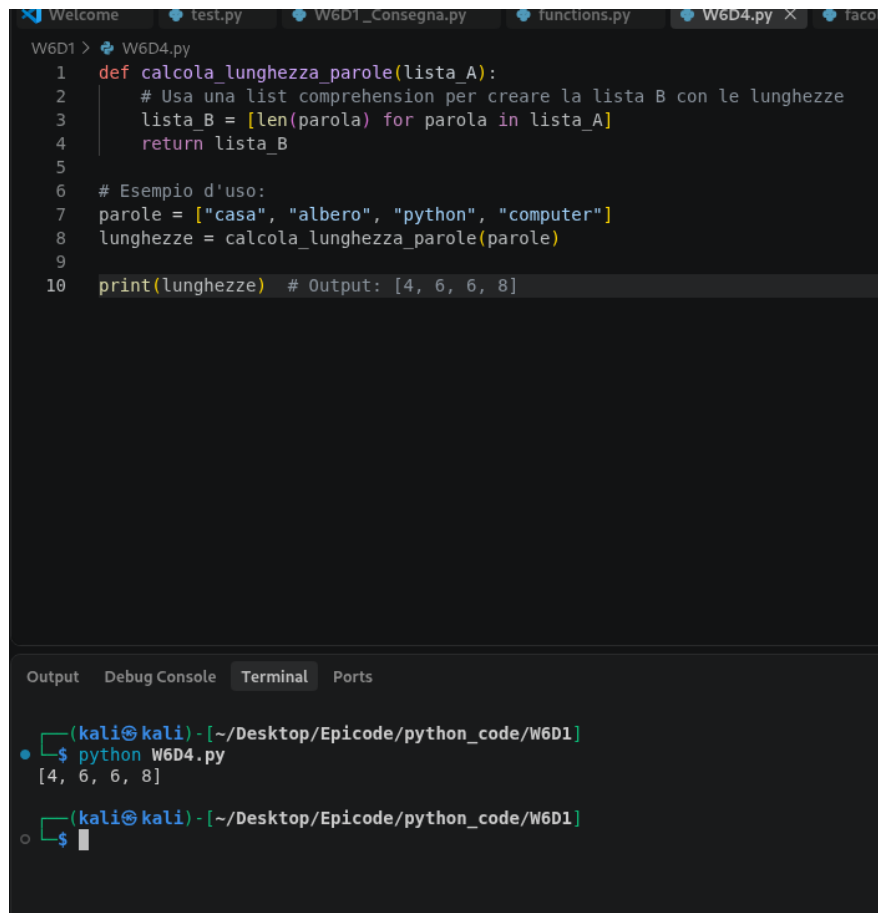
La consegna comprende due esercizi distinti:

- W6D4.py — scrivere una funzione che, data una lista di parole, restituisca una nuova lista contenente la lunghezza di ciascuna parola, utilizzando una list comprehension.
- facoltativo.py — scrivere una funzione che generi una password sicura, con due modalità: una versione semplice (8 caratteri alfanumerici) e una versione complessa (20 caratteri, comprendente lettere maiuscole/minuscole, cifre e punteggiatura).

Codice sviluppato

La funzione `calcola_lunghezza_parole(lista_A)` riceve una lista di stringhe e restituisce una nuova lista `lista_B` contenente, per ogni parola, la sua lunghezza calcolata con `len()`. La costruzione della nuova lista avviene in un'unica riga tramite list comprehension:

```
lista_B = [len(parola) for parola in lista_A]
```



```
W6D1 > W6D4.py
1 def calcola_lunghezza_parole(lista_A):
2     # Usa una list comprehension per creare la lista B con le lunghezze
3     lista_B = [len(parola) for parola in lista_A]
4     return lista_B
5
6 # Esempio d'uso:
7 parole = ["casa", "albero", "python", "computer"]
8 lunghezze = calcola_lunghezza_parole(parole)
9
10 print(lunghezze) # Output: [4, 6, 6, 8]
```

```
Output Debug Console Terminal Ports
(kali@kali) - [~/Desktop/Epicode/python_code/W6D1]
$ python W6D4.py
[4, 6, 6, 8]
(kali@kali) - [~/Desktop/Epicode/python_code/W6D1]
$
```

Codice di W6D4.py ed esecuzione da terminale (VS Code)

Test ed esecuzione

Il programma è stato eseguito con il comando `python W6D4.py` sulla lista di esempio `["casa", "albero", "python", "computer"]`.

Input (lista_A)	Risultato atteso	Risultato ottenuto	Esito
["casa", "albero", "python", "computer"]	[4, 6, 6, 8]	[4, 6, 6, 8]	OK

Il risultato ottenuto corrisponde esattamente a quello atteso: ogni elemento della lista restituita rappresenta correttamente il numero di caratteri della parola corrispondente nella lista di partenza.

Esercizio facoltativo – Generatore di password

Codice sviluppato

La funzione `genera_password(complicata=False)` utilizza i moduli `secrets` e `string` della libreria standard. Il parametro `complicata` determina il set di caratteri e la lunghezza della password generata:

- `complicata=False` → 8 caratteri, solo lettere e cifre (`string.ascii_letters + string.digits`)
- `complicata=True` → 20 caratteri, lettere, cifre e punteggiatura ASCII (`string.ascii_letters + string.digits + string.punctuation`)

La password viene costruita estraendo caratteri casuali dal set scelto con `secrets.choice()`, ripetuto per il numero di volte pari alla lunghezza richiesta, e unendo il tutto con `".join(...)"`:

```
password = ''.join(secrets.choice(caratteri) for _ in range(lunghezza))
```

L'uso del modulo `secrets`, al posto del più comune `random`, è corretto e appropriato in questo contesto: `secrets` è pensato specificamente per la generazione di valori crittograficamente sicuri (password, token), mentre `random` non garantisce l'imprevedibilità necessaria per scopi di sicurezza.

```
W6D1 > facoltativo.py
1 import secrets
2 import string
3
4 def genera_password(complicata=False):
5     if complicata:
6         # 20 caratteri: lettere (maiuscole/minuscole), cifre e punteggiatura ASCII
7         caratteri = string.ascii_letters + string.digits + string.punctuation
8         lunghezza = 20
9     else:
10        # 8 caratteri: solo lettere e cifre (alfanumerica)
11        caratteri = string.ascii_letters + string.digits
12        lunghezza = 8
13
14    # Generazione sicura della password
15    password = ''.join(secrets.choice(caratteri) for _ in range(lunghezza))
16    return password
17
18 # Esempi d'uso:
19 password_semplice = genera_password(complicata=False)
20 password_complessa = genera_password(complicata=True)
21
22 print("Password semplice (8 char):", password_semplice)
23 print("Password complessa (20 char):", password_complessa)
```

```
Output  Debug Console  Terminal  Ports
(kali@kali) - [~/Desktop/Epicode/python_code]
$ python W6D4.py
(kali@kali) - [~/Desktop/Epicode/python_code]
$ cd W6D1
(kali@kali) - [~/Desktop/Epicode/python_code/W6D1]
$ python W6D4.py
[4, 6, 6, 8]
(kali@kali) - [~/Desktop/Epicode/python_code/W6D1]
$ python facoltativo.py
Password semplice (8 char): MCCcLBVa
Password complessa (20 char): S|=reY[CFN'<8k$Af,ZE
```

Codice di `facoltativo.py` ed esecuzione da terminale (VS Code)

Test ed esecuzione

Il programma è stato eseguito con `python facoltativo.py`, generando entrambe le tipologie di password:

Tipologia	Lunghezza attesa	Password generata (esempio)
Semplice (complicata=False)	8	MCCclBVa
Complessa (complicata=True)	20	S =reY[CFN`<8k\$Af,ZE

Entrambe le password generate rispettano la lunghezza e il set di caratteri previsti dalla consegna: la versione semplice contiene solo lettere e cifre, la versione complessa include anche simboli di punteggiatura. Trattandosi di generazione casuale, il valore esatto della password cambia ad ogni esecuzione: questo è il comportamento atteso e corretto.

Problemi riscontrati

Nessun errore o comportamento anomalo è stato riscontrato durante l'esecuzione di entrambi gli script. Gli output ottenuti corrispondono in entrambi i casi a quanto previsto dalla consegna.

Conclusioni

Il primo esercizio ha permesso di applicare la list comprehension come alternativa compatta a un ciclo for con `append()`, per trasformare ogni elemento di una lista in un nuovo valore. Il secondo esercizio ha permesso di lavorare con i moduli `string` e `secrets` per la generazione controllata di stringhe casuali, distinguendo tra un caso d'uso semplice e uno a maggiore complessità/sicurezza. Entrambi gli script funzionano correttamente e non richiedono ulteriori correzioni.